

# Deep Learning

**Grafo Computacional e Programação Diferenciável**

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory  
PUCRS

maio de 2026

# Visão Geral

---

1. Definição e exemplo simples
2. Forward e backward genéricos
3. Exemplo: regressão logística
4. Programação diferenciável

# Overview

---

1. **Definição e exemplo simples**
2. Forward e backward genéricos
3. Exemplo: regressão logística
4. Programação diferenciável

# Por que pensar em grafo computacional?

---

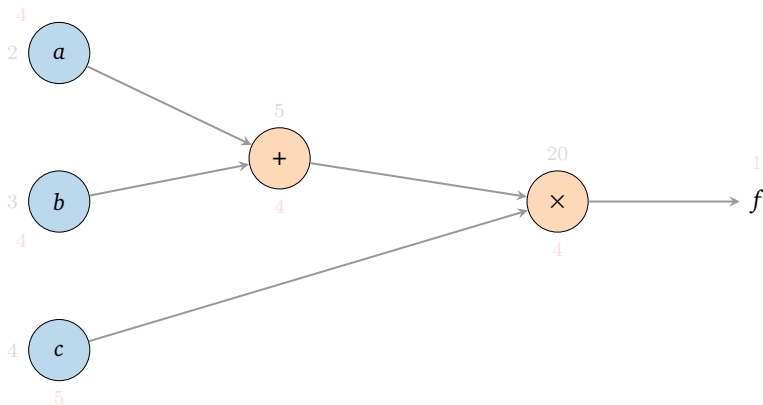
- *Backpropagation* é a aplicação repetida da regra da cadeia em uma função composta.
- Funções complicadas (uma rede profunda inteira) podem ser decompostas em operações elementares.
- Representar essa decomposição como um **grafo** torna explícito o caminho que cada gradiente local precisa percorrer.
- Mesma estrutura permite ao computador fazer a derivada **automaticamente**, ideia central de *frameworks* como PyTorch e JAX.

# Definição

---

- Um **grafo computacional** é um DAG (*directed acyclic graph*) em que:
  - nós-folha representam **variáveis** (entradas, parâmetros);
  - nós internos representam **operações** elementares (soma, produto, exp, ReLU, ...);
  - arestas indicam o fluxo de dados, das entradas em direção à saída final.
- Avaliar a função é fazer um **forward pass**: percorrer o grafo em ordem topológica.
- Calcular gradientes em relação a cada folha é fazer um **backward pass**: percorrer o grafo na ordem inversa, aplicando a regra da cadeia.

## Exemplo simples: $f(a, b, c) = (a + b) \cdot c^1$

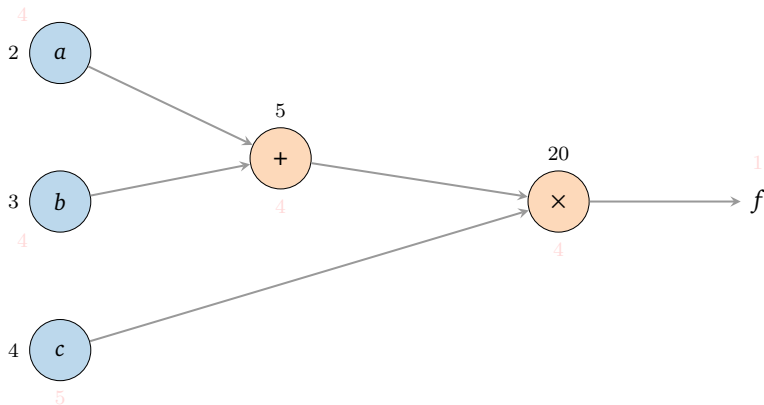


Valores em **preto** (forward), gradientes em **vermelho** (backward).

Gate  $\times$  comuta valores: gradiente recebe o valor da *outra* entrada. Gate  $+$  distribui o gradiente igualmente.

<sup>1</sup>Andrej Karpathy, Fei-Fei Li e Justin Johnson. CS231n: Convolutional Neural Networks for Visual Recognition — Backpropagation, Intuitions. <https://cs231n.github.io/optimization-2/>. Stanford University course notes. 2016.

## Exemplo simples: $f(a, b, c) = (a + b) \cdot c^1$

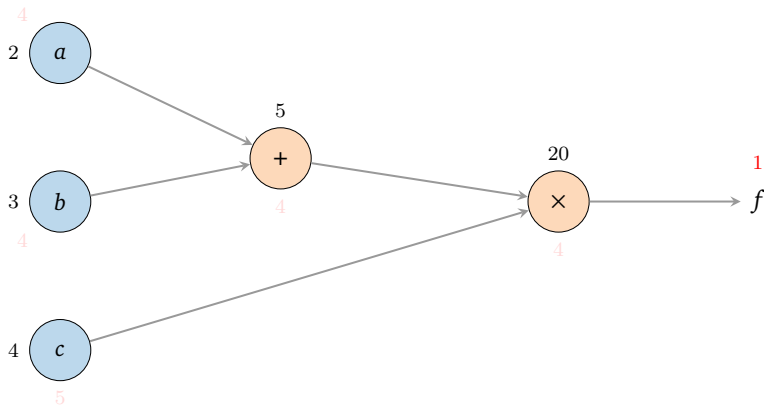


Valores em **preto** (forward), gradientes em **vermelho** (backward).

Gate  $\times$  comuta valores: gradiente recebe o valor da *outra* entrada. Gate  $+$  distribui o gradiente igualmente.

<sup>1</sup>Andrej Karpathy, Fei-Fei Li e Justin Johnson. CS231n: Convolutional Neural Networks for Visual Recognition — Backpropagation, Intuitions. <https://cs231n.github.io/optimization-2/>. Stanford University course notes. 2016.

## Exemplo simples: $f(a, b, c) = (a + b) \cdot c^1$



Valores em **preto** (forward), gradientes em **vermelho** (backward).

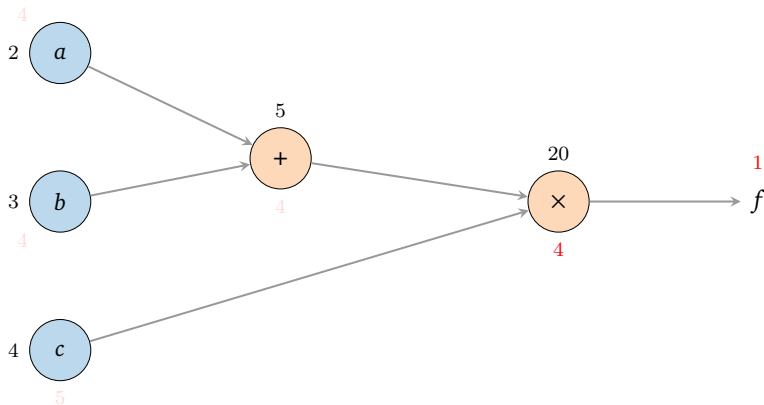
Gate  $\times$  comuta valores: gradiente recebe o valor da *outra* entrada. Gate  $+$  distribui o gradiente igualmente.

<sup>1</sup>Andrej Karpathy, Fei-Fei Li e Justin Johnson. CS231n: Convolutional Neural Networks for Visual Recognition — Backpropagation, Intuitions. <https://cs231n.github.io/optimization-2/>. Stanford University course notes. 2016.



## Exemplo simples: $f(a, b, c) = (a + b) \cdot c$ <sup>1</sup>

---

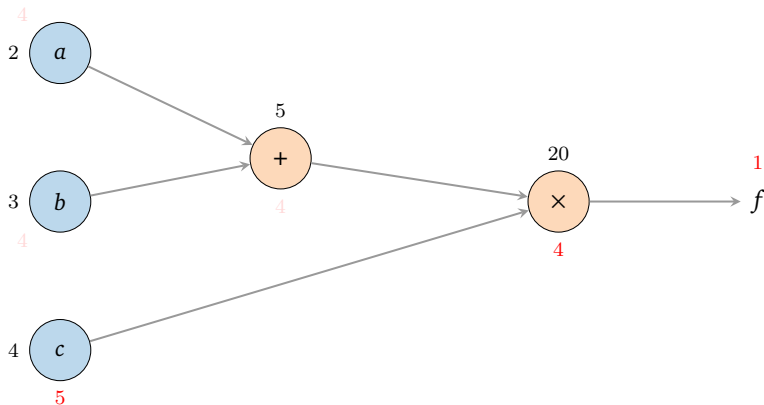


Valores em **preto** (forward), gradientes em **vermelho** (backward).  
Gate  $\times$  comuta valores: gradiente recebe o valor da *outra* entrada. Gate  $+$  distribui o gradiente igualmente.

---

<sup>1</sup>Andrej Karpathy, Fei-Fei Li e Justin Johnson. CS231n: Convolutional Neural Networks for Visual Recognition — Backpropagation, Intuitions. <https://cs231n.github.io/optimization-2/>. Stanford University course notes. 2016.

## Exemplo simples: $f(a, b, c) = (a + b) \cdot c^1$

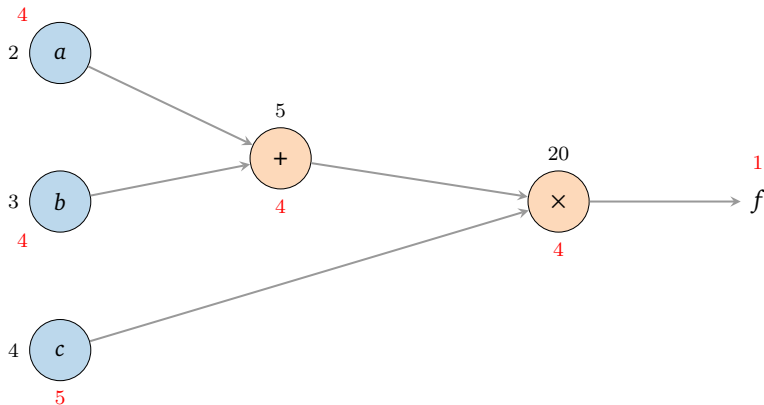


Valores em **preto** (forward), gradientes em **vermelho** (backward).  
Gate  $\times$  comuta valores: gradiente recebe o valor da *outra* entrada. Gate  $+$  distribui o gradiente igualmente.

<sup>1</sup>Andrej Karpathy, Fei-Fei Li e Justin Johnson. CS231n: Convolutional Neural Networks for Visual Recognition — Backpropagation, Intuitions. <https://cs231n.github.io/optimization-2/>. Stanford University course notes. 2016.

## Exemplo simples: $f(a, b, c) = (a + b) \cdot c$ <sup>1</sup>

---



Valores em **preto** (forward), gradientes em **vermelho** (backward).  
Gate  $\times$  comuta valores: gradiente recebe o valor da *outra* entrada. Gate  $+$  distribui o gradiente igualmente.

---

<sup>1</sup>Andrej Karpathy, Fei-Fei Li e Justin Johnson. CS231n: Convolutional Neural Networks for Visual Recognition — Backpropagation, Intuitions. <https://cs231n.github.io/optimization-2/>. Stanford University course notes. 2016.

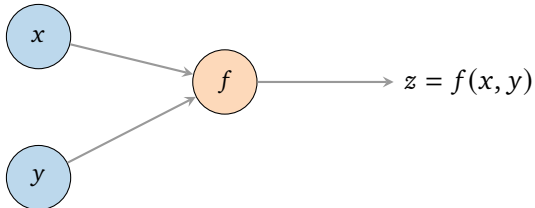
# Overview

---

1. Definição e exemplo simples
- 2. Forward e backward genéricos**
3. Exemplo: regressão logística
4. Programação diferenciável

## Um nó genérico no *forward*

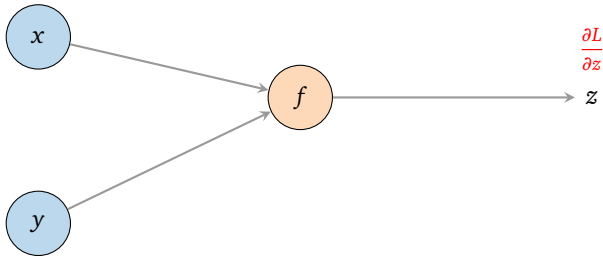
---



- No *forward*, cada nó recebe valores de entrada e calcula sua saída local.
- As saídas alimentam os nós seguintes na ordem topológica.
- No final do grafo temos o valor da função, tipicamente uma *loss* escalar  $L$ .

## O mesmo nó no *backward*

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial x}$$



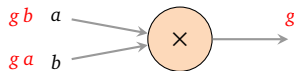
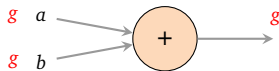
$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial y}$$

- Cada aresta no sentido oposto carrega o **gradiente upstream**  $\partial L / \partial z$ .
- O nó multiplica esse gradiente pela **derivada local**  $\partial z / \partial x$  (regra da cadeia) e envia para a aresta de entrada.
- Se uma variável aparece em múltiplos nós, os gradientes que chegam nela são **somados**.

## Padrões comuns de *gates*

---

- **Soma (+):** *distribuidor*, copia o gradiente *upstream* para todas as entradas.
- **Produto ( $\times$ ):** *comutador*, multiplica o gradiente pelo valor da *outra* entrada.
- **Máximo ( $\max$ ):** *roteador*, envia o gradiente inteiro para a entrada que venceu, zero para as demais.
- **Cópia** (uma variável usada duas vezes): *somador* no backward, soma os gradientes que chegam.



# Overview

---

1. Definição e exemplo simples
2. Forward e backward genéricos
- 3. Exemplo: regressão logística**
4. Programação diferenciável



## Outro exemplo: regressão logística

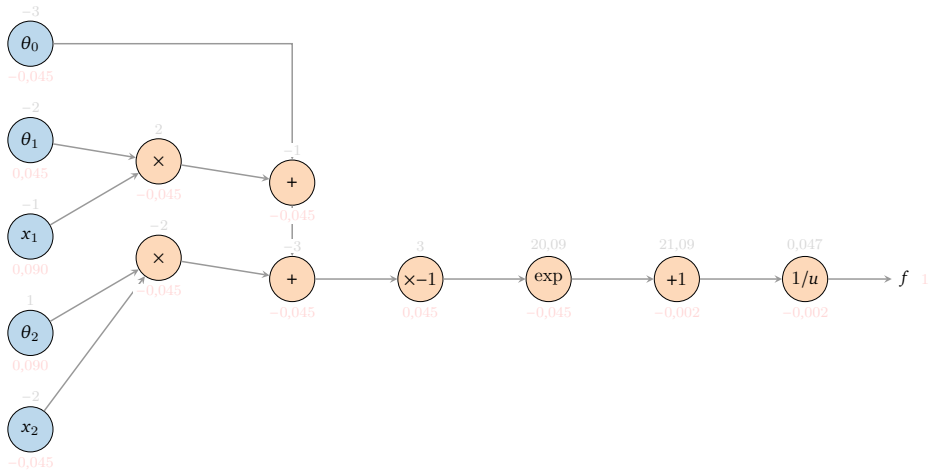
---

- Considere a função

$$f(\boldsymbol{\theta}, \mathbf{x}) = \frac{1}{1 + \exp(-(\theta_0 + \theta_1 x_1 + \theta_2 x_2))}.$$

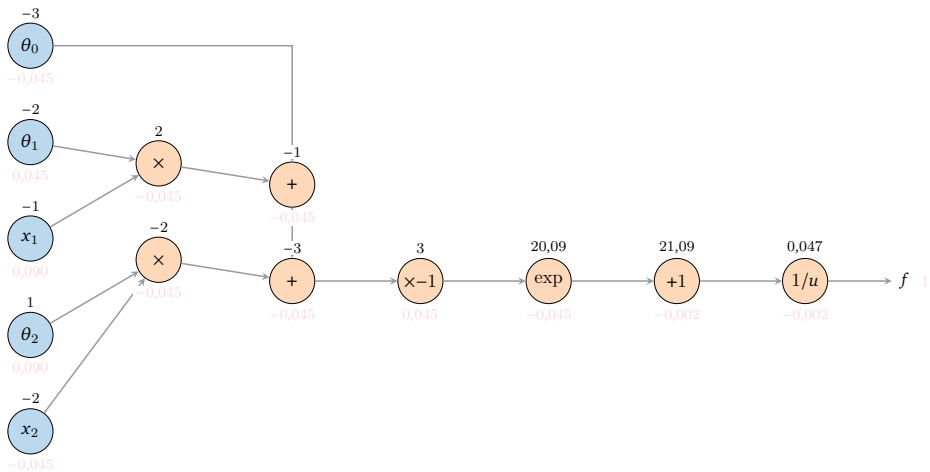
- Composição de *gates* elementares: combinação afim,  $\times(-1)$ ,  $\exp$ ,  $+1$ ,  $1/u$ .
- Vamos avaliar para  $\theta_0 = -3$ ,  $\theta_1 = -2$ ,  $\theta_2 = 1$ ,  $x_1 = -1$ ,  $x_2 = -2$ , depois rodar o backward.
- No fim, observaremos que vários *gates* podem ser agrupados em uma única operação *sigmoide*.

# Grafo da regressão logística



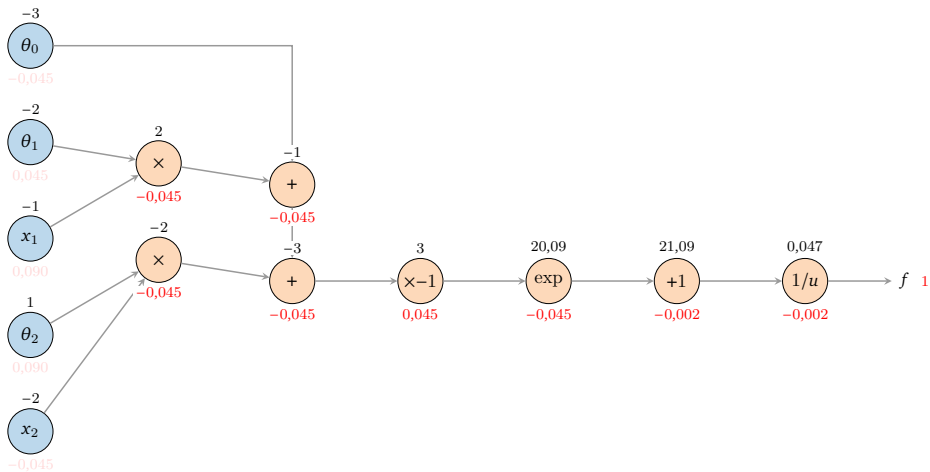
Valores em **preto** (forward), gradientes em **vermelho** (backward).

# Grafo da regressão logística



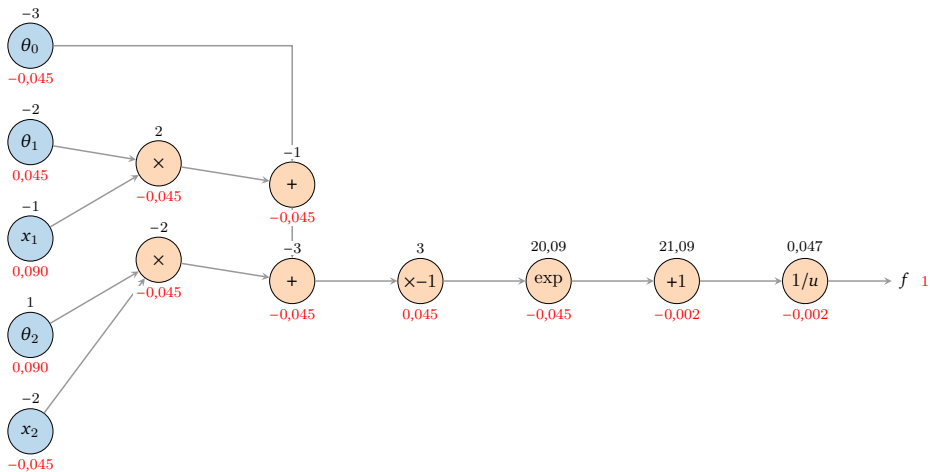
Valores em **preto** (forward), gradientes em **vermelho** (backward).

# Grafo da regressão logística



Valores em **preto** (forward), gradientes em **vermelho** (backward).

# Grafo da regressão logística



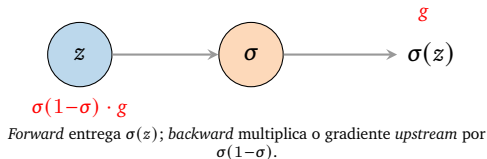
Valores em **preto** (forward), gradientes em **vermelho** (backward).

## Agrupando *gates*: a sigmoide

- A sequência  $\times -1 \rightarrow \exp \rightarrow +1 \rightarrow 1/u$  implementa  $\sigma(z) = 1/(1 + e^{-z})$ .
- Sua derivada tem forma fechada:

$$\sigma'(z) = \sigma(z) (1 - \sigma(z)).$$

- Substituir os 4 *gates* por um único nó *sigmoide* simplifica o grafo, evita instabilidades numéricas e cobra apenas uma multiplicação no *backward*.
- Granularidade fina (flexível) versus grossa (eficiente) é uma decisão recorrente em *frameworks* de *deep learning*.



# Overview

---

1. Definição e exemplo simples
2. Forward e backward genéricos
3. Exemplo: regressão logística
- 4. Programação diferenciável**

# Programação diferenciável

---

- Se cada operação elementar conhece sua derivada local, o computador pode montar o grafo *automaticamente* ao executar o código *forward*.
- Esse processo é chamado **diferenciação automática** (*autodiff*<sup>2</sup>), distinto de:
  - *symbolic* differentiation (manipulação algébrica de fórmulas);
  - *numerical* differentiation (diferença finita,  $\partial f / \partial x \approx (f(x+h) - f(x))/h$ ).
- Programação que usa *autodiff* no laço principal recebe o nome de *differentiable programming*: redes neurais são apenas um caso particular.

---

<sup>2</sup>Atilim Gunes Baydin et al. “Automatic differentiation in machine learning: a survey”. Em: [JMLR](#) 18.153 (2018), pp. 1–43.



## Forward mode vs. reverse mode

---

### Forward mode

- Propaga derivadas *junto* com o forward, da entrada para a saída.
- Custo proporcional ao número de **entradas**.
- Eficiente quando  $|entradas| \ll |saídas|$ .
- Usado em simulações físicas e análise de sensibilidade.

### Reverse mode (*backprop*)

- Faz o forward, depois propaga gradientes da saída para as entradas.
- Custo proporcional ao número de **saídas**.
- Eficiente quando  $|saídas| \ll |entradas|$ , caso típico de redes neurais (saída escalar, milhões de parâmetros).
- Padrão em *frameworks* de deep learning.

## Na prática: *autograd* de PyTorch<sup>3</sup>

---

- Cada Tensor pode ter `requires_grad=True`.
- Cada operação registrada constrói nós em um **grafo dinâmico**, reconstruído a cada *forward*.
- `loss.backward()` faz o reverse mode: percorre o grafo a partir da saída e acumula gradientes em `parameter.grad`.
- O otimizador (SGD, Adam, ...) lê esses `.grad` e atualiza os parâmetros.
- JAX, TensorFlow 2 e Flax seguem o mesmo princípio, com variações: grafos estáticos compilados, transformações de funções (`jax.grad`, `jax.jit`, `jax.vmap`).

---

<sup>3</sup>Adam Paszke et al. "PyTorch: An imperative style, high-performance deep learning library". Em: [NeurIPS](#). vol. 32. 2019.

## autograd em ação

---

```
1 import torch
2
3 a = torch.tensor(2.0, requires_grad=True)
4 b = torch.tensor(3.0, requires_grad=True)
5 c = torch.tensor(4.0, requires_grad=True)
6
7 f = (a + b) * c          # forward
8 f.backward()             # reverse mode
9
10 print(a.grad, b.grad, c.grad)
11 # tensor(4.) tensor(4.) tensor(5.)
```

- Mesmo grafo do primeiro exemplo:  $f = (a + b) \cdot c$  com  $a=2, b=3, c=4$ .
- Gradientes coincidem com a derivação manual:  
 $\partial f / \partial a = c = 4$ ,  $\partial f / \partial b = c = 4$ ,  
 $\partial f / \partial c = a + b = 5$ .
- O grafo foi montado **automaticamente** ao executar a expressão; nenhum código de derivada foi escrito.

# Leituras Recomendadas

---

- Notas de aula CS231n, módulo *Backpropagation, Intuitions*: (Karpathy, Li e Johnson, 2016), [cs231n.github.io/optimization-2](https://cs231n.github.io/optimization-2).
- Survey de *automatic differentiation*: (Baydin et al., 2018).
- Capítulo 7 de [Simon J. D. Prince](#). *Understanding Deep Learning*. MIT Press, 2023.
- PyTorch *autograd*: (Paszke et al., 2019) e documentação oficial em [pytorch.org/docs/stable/notes/autograd](https://pytorch.org/docs/stable/notes/autograd).

# Referências I

---

- [1] Atilim Gunes Baydin et al. “Automatic differentiation in machine learning: a survey”. Em: JMLR 18.153 (2018), pp. 1–43.
- [2] Andrej Karpathy, Fei-Fei Li e Justin Johnson. CS231n: Convolutional Neural Networks for Visual Recognition — Backpropagation, Intuitions. <https://cs231n.github.io/optimization-2/>. Stanford University course notes. 2016.
- [3] Adam Paszke et al. “PyTorch: An imperative style, high-performance deep learning library”. Em: NeurIPS. Vol. 32. 2019.
- [4] Simon J. D. Prince. Understanding Deep Learning. MIT Press, 2023.